

Codeforces 爬取与可视化系统

函数设计说明书（流程图在附件）

目录

- 1 系统总流程
- 2 核心函数设计
 - 2.1 main 主函数
 - 2.2 fetch_data 网络请求
 - 2.3 WriteMemoryCallback 回调接收
 - 2.4 safe_str /safe_ll 安全取值
 - 2.5 parse_user_info 解析用户信息
 - 2.6 parse_contest_records 解析比赛记录
 - 2.7 parse_ac_submissions 解析 AC 提交
 - 2.8 build_final_json 构建 JSON
 - 2.9 write_json_file 写入文件
- 3 前端可视化函数
 - 3.1 getRankClass 等级颜色
 - 3.2 drawRatingChart 绘制 Rating 曲线
 - 3.3 drawAcChart 绘制补题饼图
 - 3.4 fillContestTable /fillProblemTable 表格渲染

2.1 main（主程序入口）

所在文件：cf.c

```
int main() { // 初始化缓冲区
curl_memset(buf1, 0, BUF_SIZE);
    memset(buf2, 0, BUF_SIZE);
memset(buf3, 0, BUF_SIZE);
    curl_global_init(CURL_GLOBAL_ALL);
const char *handle = "Nexus_Raphael";
    // 1. 爬取用户基本信息
fetch_user_info(handle);
    // 2. 爬取比赛 rating 记录
fetch_contest_records(handle);
    // 3. 爬取 AC 提交记录
fetch_ac_submissions(handle);
    // 4. 解析并生成 JSON
cJSON *final = build_final_json(handle);
    // 5. 写入文件
write_json_file(final);
    // 释放资源
```

```
cJSON_Delete(final); curl_global_cleanup(); return 0; }
```

2.2 fetch_data (HTTP 请求函数)

所在文件: cf.c

```
void fetch_url(const char *url, char *buf, size_t buf_size) {  
  
    CURL *c = curl_easy_init();  
  
    curl_easy_setopt(c, CURLOPT_URL, url);  
  
    curl_easy_setopt(c, CURLOPT_WRITEFUNCTION, write_callback);  
  
    curl_easy_setopt(c, CURLOPT_WRITEDATA, buf);  
  
    curl_easy_setopt(c, CURLOPT_SSL_VERIFYPEER, 0L);  
  
    curl_easy_setopt(c, CURLOPT_FOLLOWLOCATION, 1L);  
  
  
    curl_easy_perform(c);  
  
    curl_easy_cleanup(c);  
}
```

2.3 WriteMemoryCallback (数据接收回调)

所在文件: cf.c

```
size_t cb1(void *data, size_t size, size_t nmemb, char *userp) {  
  
    size_t len = size * nmemb;  
  
    if (strlen(buf1) + len < BUF_SIZE - 1) {  
  
        strncat(buf1, (char*)data, len);  
  
    }  
  
    return len; }  
  
size_t cb2(...); // 对应 buf2size_t cb3(...); // 对应 buf3
```

2.4 safe_str /safe_ll

所在文件: cf.c

```

char* safe_str(cJSON *obj, const char *key, char *def) {

    if (!obj) return def;

    cJSON *item = cJSON_GetObjectItem(obj, key);

    if (item && cJSON_IsString(item))

        return item->valuelstring;

    return def;}

long long safe_ll(cJSON *obj, const char *key);

```

2.5 parse_user_info (解析用户信息)

所在文件: cf.c

// 1.用户基本信息

```

CURL *c1 = curl_easy_init();char url1[256];sprintf(url1,
"https://codeforces.com/api/user.info?handles=%s",
handle);curl_easy_setopt(c1, CURLOPT_URL, url1);curl_easy_setopt(c1,
CURLOPT_WRITEFUNCTION, cb1);curl_easy_perform(c1);curl_easy_cleanup(c1);

cJSON *root1 = cJSON_Parse(buf1);
cJSON *result1 = cJSON_GetObjectItem(root1, "result");
cJSON *user = cJSON_GetArrayItem(result1, 0);

char *rank = safe_str(user, "rank", "无段位");int rating = (int)safe_ll(user,
"rating");char *maxRank = safe_str(user, "maxRank", "无段位");int maxRating =
(int)safe_ll(user, "maxRating");char *avatar = safe_str(user, "titlePhoto", "");

```

2.6 parse_contest_records (解析比赛记录)

所在文件: cf.c

// 2.比赛记录

```

CURL *c2 = curl_easy_init();char url2[256];sprintf(url2,
"https://codeforces.com/api/user.rating?handle=%s",
handle);curl_easy_setopt(c2, CURLOPT_URL, url2);curl_easy_setopt(c2,
CURLOPT_WRITEFUNCTION, cb2);curl_easy_setopt(c2, CURLOPT_SSL_VERIFYPEER,
0L);curl_easy_setopt(c2, CURLOPT_FOLLOWLOCATION,
1L);curl_easy_perform(c2);curl_easy_cleanup(c2);

```

```

cJSON *root2 = cJSON_Parse(buf2);

cJSON *contests = cJSON_GetObjectItem(root2, "result"); int contestCount =
cJSON_GetArraySize(contests);

int recent180Count = 0; time_t now = time(NULL); const long long day180Sec =
15552000LL; for (int i = 0; i < contestCount; i++) {

    cJSON *one = cJSON_GetArrayItem(contests, i);

    long long t = safe_ll(one, "ratingUpdateTimeSeconds");

    if ((now - t) <= day180Sec) {

        recent180Count++;

    }}

cJSON *records = cJSON_CreateArray(); for (int i = 0; i < contestCount; i++) {

    cJSON *one = cJSON_GetArrayItem(contests, i);

    cJSON *item = cJSON_CreateObject();

    cJSON_AddStringToObject(item, "name", safe_str(one, "contestName", "未知比
赛"));

    cJSON_AddNumberToObject(item, "time", safe_ll(one,
"ratingUpdateTimeSeconds"));

    cJSON_AddNumberToObject(item, "oldRating", safe_ll(one, "oldRating"));

    cJSON_AddNumberToObject(item, "newRating", safe_ll(one, "newRating"));

    cJSON_AddNumberToObject(item, "rank", safe_ll(one, "rank"));

    cJSON_AddItemToArray(records, item);}

```

2.7 解析 AC 提交

所在文件: cf.c

```
// 3.AC 提交、补题记录
```

```

CURL *c3 = curl_easy_init(); char url3[256]; sprintf(url3,
"https://codeforces.com/api/user.status?handle=%s&from=1&count=1000",
handle); curl_easy_setopt(c3, CURLOPT_URL, url3); curl_easy_setopt(c3,
CURLOPT_WRITEFUNCTION, cb3); curl_easy_setopt(c3, CURLOPT_SSL_VERIFYPEER,
0L); curl_easy_setopt(c3, CURLOPT_FOLLOWLOCATION,
1L); curl_easy_perform(c3); curl_easy_cleanup(c3);

```

```

cJSON *root3 = cJSON_Parse(buf3);

cJSON *submissions = cJSON_GetObjectItem(root3, "result"); int subCount =
cJSON_GetArraySize(submissions);

cJSON *acProblems = cJSON_CreateArray(); int acTotal = 0; for (int i = 0; i < subCount;
i++) {

    cJSON *sub = cJSON_GetArrayItem(submissions, i);

    cJSON *verdict = cJSON_GetObjectItem(sub, "verdict");

    if (verdict && strcmp(verdict->valuelstring, "OK") == 0) {

        acTotal++;

        cJSON *prob = cJSON_GetObjectItem(sub, "problem");

        cJSON *pItem = cJSON_CreateObject();

        cJSON_AddNumberToObject(pItem, "contestId", safe_ll(prob,
"contestId"));

        cJSON_AddStringToObject(pItem, "index", safe_str(prob, "index", ""));

        cJSON_AddStringToObject(pItem, "name", safe_str(prob, "name", ""));

        cJSON_AddNumberToObject(pItem, "time", safe_ll(sub,
"creationTimeSeconds"));

        cJSON_AddItemToArray(acProblems, pItem);

    }
}

```

2.8 build_final_json

所在文件: cf.c

```

cJSON *build_final_json(const char *handle) {

    cJSON *final = cJSON_CreateObject();

    cJSON_AddStringToObject(final, "handle", handle);

    cJSON_AddStringToObject(final, "currentRank", user.rank);

    cJSON_AddNumberToObject(final, "currentRating", user.rating);

    cJSON_AddItemToObject(final, "records", contest_array);
}

```

```
cJSON_AddItemToObject(final, "acProblems", ac_array);
```

```
return final;}
```

2.9 write_json_file

所在文件: cf.c

```
FILE *fp = fopen("data.json", "w");
```

```
char *str = cJSON_Print(root);
```

```
fprintf(fp, "%s", str);
```

```
fclose(fp);
```

```
free(str);
```

3.1getRankClass (等级颜色映射)

所在文件: cf.html

```
function getRankClass(rating) {  
  
    if (rating < 1200) return 'newbie';  
  
    if (rating < 1400) return 'pupil';  
  
    if (rating < 1600) return 'specialist';  
  
    if (rating < 1900) return 'expert';  
  
    if (rating < 2100) return 'candidate';  
  
    if (rating < 2300) return 'master';  
  
    return 'grandmaster';}
```

3.2drawRatingChart (绘制 Rating 曲线)

所在文件: cf.html (内嵌 script 中)

```
function drawRatingChart(list){  
  
    let dateArr = [];  
  
    let ratingArr = [];  
  
    let nameArr = [];
```

```

list.forEach(item=>{

    let d = new Date(item.time * 1000);

    dateArr.push(`${d.getFullYear()}-${d.getMonth()+1}-${d.getDate()}`);

    ratingArr.push(item.newRating);

    nameArr.push(item.name);

});

const option = {

    tooltip: {

        trigger: 'axis',

        formatter: params => {

            const idx = params[0].dataIndex;

            return `比赛: ${nameArr[idx]}<br>日期: ${dateArr[idx]}<br>Rating:
${ratingArr[idx]}`;

        }

    },

    xAxis: {

        type: 'category',

        data: dateArr,

        axisLabel: { rotate: 45 }

    },

    yAxis: {

        type: 'value',

        name: 'Rating'

    },

    series: [{

        name: 'Rating',

        type: 'line',

        data: ratingArr,

        smooth: true,

```

```

        lineStyle: { width: 3, color: '#FF0000' },

        itemStyle: { color: '#FF0000' }

    }]

};

ratingChart.setOption(option);}

```

3.3 drawAcChart (绘制补题分布饼图)

所在文件: cf.html

```

function drawAcChart(contests, problems){

    let contestAc = 0;

    let afterAc = 0;

    problems.forEach(p => {

        let isContest = contests.some(c => {

            let contestStart = c.time * 1000;

            let acTime = p.time * 1000;

            let isDiv3 = /Div\.\?\s*3/i.test(c.name);

            let duration = isDiv3 ? 8100 : 7200;

            let threshold = duration * 1000;

            return Math.abs(acTime - contestStart) < threshold;

        });

        if(isContest) contestAc++;

        else afterAc++;

    });

    const option = {

        tooltip: { trigger: 'item' },

        legend: { orient: 'vertical', left: 'left' },

        series: [{

```



```

        type: 'pie',

        radius: ['40%', '70%'],

        avoidLabelOverlap: false,

        itemStyle: { borderRadius: 10, borderColor: '#fff', borderWidth: 2 },

        label: { show: false, position: 'center' },

        emphasis: {

            label: { show: true, fontSize: 20, fontWeight: 'bold' }

        },

        labelLine: { show: false },

        data: [

            { value: contestAc, name: '比赛中 AC', itemStyle: { color:

'#008000' } },

            { value: afterAc, name: '赛后补题', itemStyle: { color: '#0066cc' } }

        ]

    }

}

acChart.setOption(option);

```

3.4 fillContestTable / fillProblemTable

所在文件: cf.html

```

function fillContestTable(list){

    const tbody = document.getElementById('contest-tbody');

    tbody.innerHTML = "";

    list.forEach(item => {

        let d = new Date(item.time * 1000);

        let dateStr = `${d.getFullYear()}-${d.getMonth()+1}-${d.getDate()}`;

        let diff = item.newRating - item.oldRating;

        let diffStr = diff >= 0 ? "+" + diff : diff;
    });
}

```

```

    let tr = document.createElement("tr");

    tr.innerHTML = `

        <td>${item.name}</td>

        <td>${dateStr}</td>

        <td class="${getRankClass(item.oldRating)}">${item.oldRating}</td>

        <td class="${getRankClass(item.newRating)}">${item.newRating}</td>

        <td>${item.rank}</td>

        <td>${diffStr}</td>

    `;

    tbody.appendChild(tr);

});}

//fillProblemTable

function fillProblemTable(contests, problems){

    const tbody = document.getElementById('problem-tbody');

    tbody.innerHTML = "";

    problems.forEach(p => {

        let acDate = new Date(p.time * 1000);

        let dateStr =

`${acDate.getFullYear()}-${acDate.getMonth()+1}-${acDate.getDate()}`;

        let isAfterContest = !contests.some(c => {

            let contestStart = c.time * 1000;

            let acTime = p.time * 1000;

            let isDiv3 = /Div\.\?\s*3/i.test(c.name);

            let duration = isDiv3 ? 8100 : 7200;

            let threshold = duration * 1000;

            return Math.abs(acTime - contestStart) < threshold;

        });

```

```

let tr = document.createElement("tr");

tr.innerHTML = `

    <td>${p.contestId}${p.index}</td>

    <td>${p.name}</td>

    <td>${dateStr}</td>

    <td class="solved">已 AC</td>

    <td class="${isAfterContest ? 'after-contest' :
'solved'}">${isAfterContest ? '是（赛后补题）' : '否（比赛中提交）'}</td>

`;

tbody.appendChild(tr);

});}

```